

第三章 软件测试概述

一、单选题

1. 软件测试越早开始越好，因此软件测试应从()阶段开始。（单选题）

- A. 产品发布
- B. 需求分析(编制产品说明书)
- C. 设计
- D. 编码

答案：B

解析：软件工程实践表明缺陷发现越晚修复成本越高。选项 B 正确，软件测试应该从需求分析阶段就开始介入——测试人员参与需求评审，检查需求的完整性、一致性和可测试性，从源头预防缺陷。如果在需求阶段就发现一个错误，修复成本可能只需 1 个单位；到编码阶段修复可能需要 10 个单位；到发布后再修复则可能高达 100 个单位。选项 A 错误，等到产品发布再测试为时已晚；选项 C 和 D 错误，设计阶段或编码阶段才开始测试会错失早期发现需求缺陷的机会。这就是“尽早测试”原则——测试活动应与开发活动并行展开。

2. 下列不属于测试用例设计原则的是()（单选题）

- A. 测试用例的测试结果具有可再现性
- B. 测试用例的测试结果不具有可再现性
- C. 测试用例的测试结果具有可判定性
- D. 测试用例应具有代表性

答案：B

解析：测试用例设计需遵循多项基本原则。选项 B 不属于测试用例设计原则，恰恰相反——测试用例的结果必须具有“可再现性”，即相同的测试用例在相同条件下多次执行应得到一致的结果。如果测试结果不可再现，就无法判断发现的现象是软件

缺陷还是环境偶然因素。选项 A 正确，可再现性是测试用例的基本要求；选项 C 正确，可判定性意味着测试用例必须有明确的预期结果，执行后能判断通过与否；选项 D 正确，测试用例应覆盖典型场景和边界情况，具有代表性。

3. 软件测试的目的是()。(单选题)

- A. 发现程序中的所有错误
- B. 调试程序
- C. 尽可能多地发现程序中的错误
- D. 证明程序是正确的

答案：C

解析：这道题考查对测试目标的理性认识。选项 C 正确，软件测试的现实目标不是也不可能发现所有错误，而是“在给定的时间、资源和成本约束下，尽可能多地发现重要错误”。选项 A 错误，穷尽所有错误是不可能的——对于稍复杂的程序，其输入域和路径组合是天文数字，无法完全覆盖；选项 B 错误，调试（消除错误）和测试（发现错误）是不同的活动；选项 D 错误，测试不能证明程序正确，只能证明有错误存在。测试人员的思维方式应从“证明正确”转变为“发现错误”。

4. 关于软件测试，以下说法错误的是()。(单选题)

- A. 80%的缺陷聚集在 20%的模块中，经常出错的模块改错后还会经常出现
- B. 开发人员测试自己的程序后，可作为该程序已经通过测试的依据
- C. 测试能提高软件的质量，但是提高质量不能依赖测试
- D. 测试只能证明缺陷存在，不能证明缺陷不存在

答案：B

解析：本题考查软件测试的基本原则。选项 B 错误，开发人员测试自己的程序不能作为正式通过的充分依据，原因有三：（1）开发者存在“思维盲区”——设计代码时的思维定式会延续到测试中，难以发现自己的逻辑错误；（2）可能潜意识里回

避对自己代码的“破坏性”测试；（3）独立测试（由专门的测试人员或 QA 团队执行）是行业公认的最佳实践。选项 A 正确，这描述的是软件缺陷的“二八定律”（Pareto 原则）——少数模块集中了多数缺陷；选项 C 正确，高质量需要需求管理、设计评审、配置管理等多方面共同保障；选项 D 正确，这是测试理论的基本认识。

5. 软件测试类型按开发阶段划分为（ ）。 （单选题）

- A. 单元测试、集成测试、系统测试、验收测试
- B. 调试、单元测试、功能测试
- C. 单元测试、集成测试、确认测试
- D. 需求测试、单元测试、集成测试

答案：A

解析：软件测试按开发阶段依次分为四个层级，形成经典的“V 模型”测试体系。选项 A 正确：（1）单元测试——针对最小可测试单元（函数、类、模块），由开发人员执行，验证代码逻辑正确性；（2）集成测试——验证模块间的接口和交互，可采用自顶向下、自底向上或三明治集成策略；（3）系统测试——在完整的系统环境中验证功能和非功能需求（性能、安全、兼容性等）；（4）验收测试——由用户或客户主导，确认软件是否满足业务需求和合同约定。选项 B 将“调试”混入测试类型；选项 C 遗漏了验收测试；选项 D 中“需求测试”不是标准的阶段划分术语。

6. 软件测试是采用（ ）执行软件的活动。 （单选题）

- A. 测试环境
- B. 输入数据
- C. 输入条件
- D. 测试用例

答案：D

解析：选项 D 正确，软件测试是通过设计和执行“测试用例”（Test Case）来进行的有计划的活动。一个完整的测试用例通常包括：用例编号、测试标题、前置条件、测试输入数据、操作步骤、预期结果、实际结果、测试状态等要素。测试用例是测试工作的核心载体，它使得测试变得系统化、可管理和可重复。选项 A 测试环境是执行测试的软硬件配置，是条件而非执行手段；选项 B 输入数据只是测试用例的一个组成部分；选项 C 输入条件同样只是测试用例的要素之一。测试用例将输入、操作、预期结果整合为一个有机整体。

7. 以下（ ）不属于黑盒测试方法的优势。（单选题）

- A. 与软件的内部实现无关
- B. 从用户的角度出发，能很容易知道用户会用到哪些功能
- C. 比较简单，不需要了解程序内部的代码及实现
- D. 不可能覆盖所有的代码，覆盖率较低

答案：D

解析：本题要求选出“不是优势”的选项。选项 D 描述的是黑盒测试的劣势而非优势——由于黑盒测试不关注代码内部逻辑，仅通过输入输出来验证功能，它天然无法穷尽所有代码路径，代码覆盖率通常低于白盒测试。选项 A 是优势，黑盒测试不依赖内部实现，即使代码重构测试用例依然有效；选项 B 是优势，黑盒测试从用户视角出发，容易覆盖用户实际使用场景；选项 C 是优势，黑盒测试不需要编程知识，业务人员和初级测试人员也可参与。黑盒与白盒各有优劣，实际项目中两者应结合使用以取长补短。

三、判断题

8. 穷尽测试是不可能的。（判断题）

- A. 对
- B. 错

答案：A

解析：此题说法正确（选A）。穷尽测试（Exhaustive Testing）指对所有可能的输入值、执行路径和状态组合进行全面测试。对于任何非平凡的程序，穷尽测试在数学上是不可能的。例如：一个要求输入两个 32 位整数的方法，其合法输入域就有 2^{64} 种组合（约 1800 亿亿种），即使每秒测试 100 万个用例，也需要 58 万年才能穷尽。这就是为什么测试需要采用等价类划分、边界值分析等技术来用有限用例代表无限可能，以及为什么测试永远不能说“程序没有错误”。

9. 白盒测试又称基于程序的测试。（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。白盒测试（White-Box Testing）也称为基于程序的测试或结构化测试。其核心特征是测试者了解并基于程序内部结构和实现逻辑来设计测试用例。白盒测试关注代码的内部细节：语句、分支、条件、路径等逻辑结构的覆盖情况。常见的白盒覆盖标准包括语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖和路径覆盖，强度依次递增。白盒测试通常在单元测试阶段由开发人员进行。

10. 测试工作具有创造性。（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。很多人误以为测试只是机械地执行测试用例，实际上优秀的测试工作高度依赖创造力。创造性体现在多个方面：（1）设计高效且能暴露隐藏错误的测试用例需要洞察力和想象力；（2）探索性测试中，测试人员基

于实时观察灵活调整测试策略，类似于侦探破案；（3）错误推测法中，测试者基于经验和直觉猜想程序可能在哪些情况下出错；（4）构建复杂的测试场景和测试数据需要巧妙的设计。优秀的测试人员不仅是“验证者”，更是“质疑者”和“探索者”。

11. 黑盒测试又称基于规格说明的测试。（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。黑盒测试不关心程序内部结构和代码实现，只依据软件的需求规格说明书来验证功能是否正确。测试人员将程序视为一个“不能打开的黑盒子”，仅通过输入和输出来评估软件行为是否符合规格。常见的黑盒测试方法包括等价类划分、边界值分析、因果图法、判定表、正交实验法和场景法等。黑盒测试侧重于“软件应该做什么”，适合功能测试、系统测试和验收测试阶段。

12. 在任何情况下执行回归测试时，都应将以前测试过的用例全部执行一遍。（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选B）。回归测试的目的是验证代码修改（如修复缺陷、新增功能）后原有功能是否仍然正常。虽然理想情况下应重新运行所有测试用例，但在实际项目中这往往不可行——测试套件可能包含成千上万个用例，全量执行耗时过长。实践中通常采用“选择性回归测试”策略：基于风险评估和修改影响范围，优先执行核心功能用例、与修改模块相关的用例、以及历史上频繁出错的用例。测试用例的优先级划分和测试套件的持续优化是回归测试管理的重要课题。

13. 测试旨在防止错误的发生。（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选B）。测试的主要目的是“发现”已经存在的错误，而非“防止”错误的发生。防止错误属于质量保证（QA）的范畴：通过制定开发规范、进行设计评审、代码审查、培训开发人员等预防性活动来减少缺陷的引入。测试是质量控制（QC）的手段，属于检测性活动——它在错误引入后进行排查和发现。二者关系可以类比为：质量保证是“建造高质量的围栏”，测试是“清点逃出去的羊”。不过，当测试发现的缺陷被分析根因并反馈到开发过程中，测试也间接起到了预防未来类似错误的作用。

14. 测试用例应由测试输入数据和对应的实际输出结果这两部分组成。（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选B）。一个完整的测试用例至少应包含以下要素：（1）测试输入数据——驱动测试执行的数据；（2）操作步骤——如何执行测试；（3）预期输出结果——根据需求规格确定的理论正确结果。题目中说“实际输出结果”是不对的——实际输出是测试执行后才得到的，而测试用例应在执行前设计好，包含的是“预期结果”。只有明确了预期结果，执行后才能将实际输出与预期输出进行比较，判定测试通过与否。此外，完善的测试用例还应包含前置条件、测试环境要求等要素。

15. 白盒测试又称逻辑驱动测试。（判断题）

A. 对

B. 错

答案: A

解析: 此题说法正确 (选 A)。白盒测试也称为逻辑驱动测试 (Logic-Driven Testing)，因为它是基于程序的内部逻辑结构来驱动测试用例设计的。测试人员根据代码的控制流 (if-else、循环、switch 等) 和逻辑条件来设计测试数据，目标是覆盖尽可能多的逻辑路径。与之相对，黑盒测试称为数据驱动测试，因为它是基于输入输出数据来驱动测试的。白盒测试的核心覆盖标准 (语句覆盖、判定覆盖、条件覆盖等) 本质上都是对程序逻辑不同粒度的覆盖要求。

16. 黑盒测试又称数据驱动测试。(判断题)

A. 对

B. 错

答案: A

解析: 此题说法正确 (选 A)。黑盒测试也称为数据驱动测试 (Data-Driven Testing)，因为测试是通过输入各种数据、检查输出来验证功能的。测试人员不需要了解内部代码逻辑，主要关注“对于给定的输入数据，软件是否产生正确的输出”。数据驱动测试的思想也与自动化测试中的数据驱动框架 (DDT) 一脉相承——将测试数据与测试脚本分离，通过不同的数据组合来驱动同一测试逻辑反复执行。这与白盒测试的“逻辑驱动”形成了鲜明的对称。

17. 单元测试通常由开发人员进行。(判断题)

A. 对

B. 错

答案: A

解析: 此题说法正确 (选 A)。单元测试是对软件中最小的可测试单元 (如 Java 中的方法、C 中的函数、C++ 中的类) 进行验证的活动，通常由编写该代码的开发

人员负责。原因包括：（1）开发人员最了解自己代码的内部逻辑和预期行为；（2）单元测试往往需要借助测试框架（如 JUnit、pytest）编写测试代码，本身就是编程工作；（3）单元测试通常在编码完成后立即执行，属于开发人员的日常工作流程。但实践中也应注意避免“开发人员自测即可”的误区——代码作者应编写单元测试，但不应是唯一的测试执行者，独立测试同样重要。

18. 白盒测试又称结构测试。（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选 A）。白盒测试也被称为结构测试（Structural Testing），因为它基于程序的内部结构来设计和执行测试。结构测试关注代码的“骨架”——控制流结构（顺序、分支、循环）和数据流结构（变量的定义和使用路径）。与之对应的是黑盒测试的功能测试视角——关注软件“应该做什么”而非“内部怎么构建的”。结构测试的常见技术包括控制流测试（语句覆盖、分支覆盖、路径覆盖）和数据流测试（定义-使用覆盖）。理解这两个视角的互补性是做好全面测试的关键。